

Functional Interfaces (Core of Java 8)

A **functional interface** has exactly **one abstract method**.

Common functional interfaces:

- **Runnable** – run()
- **Callable** – call()
- **Comparable** – compareTo()
- **Comparator** – compare()
- **Supplier<T>** – returns value
- **Consumer<T>** – accepts value
- **Function<T,R>** – transforms value
- **Predicate<T>** – boolean condition

Shortcut rule:

If you can use a lambda with it → it's a functional interface.

Lambda Expressions

Used to write short functions.

Format:

(parameters) -> { body }

Examples:

```
() -> System.out.println("Hello")  
x -> x * x  
(a, b) -> a + b
```

When used?

- Sorting
- Filtering
- Iterating
- Thread creation
- Removing boilerplate code

Method References (Shortcut of Lambda)

Syntax:

```
ClassName::methodName
```

Examples:

```
System.out::println  
String::toUpperCase  
Integer::parseInt
```

Types:

- **Static method reference** → Class::method
- **Instance method reference** → obj::method
- **Constructor reference** → Class::new

Stream API

A **stream** processes data in a pipeline.

Steps:

1. **Create stream:**
list.stream()
2. **Process:**
.filter(), .map(), .sorted()
3. **Output:**
.collect(), .forEach()

Popular methods:

- **filter()** – keeps elements
- **map()** – transform
- **flatMap()** – flatten nested structures
- **sorted()** – sort elements
- **distinct()** – remove duplicates
- **limit()** / **skip()**
- **findFirst()**, **findAny()**
- **anyMatch()**, **allMatch()**, **noneMatch()**

- **reduce()** – combine elements
- **collect()** – convert to list/set/map

Stream Examples (Must Remember)

Filter even numbers:

```
list.stream().filter(i -> i % 2 == 0).toList();
```

Convert to uppercase:

```
list.stream().map(String::toUpperCase).toList();
```

Find max:

```
list.stream().max(Integer::compare).get();
```

Count:

```
list.stream().filter(i -> i > 5).count();
```

Convert List to Set:

```
list.stream().collect(Collectors.toSet());
```

Convert to Map:

```
list.stream().collect(Collectors.toMap(User::getId, User::getName));
```

Optional

Optional is used to avoid NullPointerException.

Create:

```
Optional.of(value)  
Optional.ofNullable(value)  
Optional.empty()
```

Use:

```
opt.orElse(defaultValue)  
opt.orElseGet(() -> "default")  
opt.ifPresent(x -> System.out.println(x))
```

Most asked line:

Optional is a container that may or may not hold a value. Used to avoid null checks.

Date & Time API (Replaces Old Date)

Major classes:

- **LocalDate** – only date
- **LocalTime** – only time
- **LocalDateTime** – both
- **ZonedDateTime** – timezone
- **DateTimeFormatter** – format/parse

Examples:

```
LocalDate.now();  
LocalTime.now();  
LocalDateTime.now();
```

Formatting:

```
LocalDate.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy"));
```

Default & Static Methods in Interfaces

Before Java 8:

Interfaces → only abstract methods.

After Java 8:

Interfaces can have:

- **default methods**
- **static methods**

Example:

```
interface A {  
    default void show() { }  
    static void print() { }  
}
```

forEach() – Most Used Utility Method

```
list.forEach(System.out::println);
```

Works on List, Set, Map (with entrySet).

Collectors – Convert Stream Output

Useful collectors:

- `toList()`
- `toSet()`
- `joining(",")`
- `groupingBy()`
- `partitioningBy()`
- `counting()`
- `summingInt()`

Example:

```
map = list.stream()
           .collect(Collectors.groupingBy(Function.identity(),
           Collectors.counting()));
```

Parallel Streams

Used for multi-core processing.

```
list.parallelStream().filter(i -> i > 5).toList();
```

Warning:

Not recommended for small tasks → overhead is high.

Stream vs Collections (One-Liner)

Collections	Streams
Stores data	Processes data
Reusable	Not reusable
External iteration	Internal iteration
Modifies original	Does not modify

Top 10 Java 8 Interview Points (Quickest Revision)

- Lambda = anonymous function
- Stream = process data in pipeline
- Optional = avoid null

- Method Ref = shorter lambda
- Functional Interface = 1 abstract method
- Interface can have default methods
- Date/Time API → clean and immutable
- Collectors used for grouping/summarizing
- `parallelStream()` for multi-core
- `map()` transforms, `filter()` removes

JAVA Interview Buddy